

AWS Deployment Guide — Fresh EC2 to Live Production

A complete, step-by-step walkthrough for taking the AFS Financial CRM from a brand-new Ubuntu EC2 instance to a live, SSL-secured production deployment. Written so any future team member can follow it start to finish without prior context.

Assumes: a fresh Ubuntu 22.04 LTS (or 24.04 LTS) EC2 instance, an AWS account with permission to create EC2/Security Group/Elastic IP resources, and a domain name you can point DNS for. Commands are written for that instance's own SSH session unless noted otherwise. For the pre-flight/post-flight checklist form of this same information, see the companion **AWS Deployment Checklist**.

01 Launch the EC2 instance

In the AWS Console: EC2 → Launch Instance.

- **AMI:** Ubuntu Server 22.04 LTS (or 24.04 LTS), 64-bit (x86_64).
- **Instance type:** `t3.medium` (2 vCPU / 4 GB) as a practical minimum — Mongo, Redis, the API process, and the worker process all run on this one box. Go to `t3.large` if you expect meaningful concurrent staff usage from day one.
- **Storage:** at least 30 GB gp3 — MongoDB data, logs, and the Node/Python toolchains add up quickly.
- **Key pair:** create a new one and download the `.pem` file. You cannot download it again later — store it somewhere safe (password manager, not email).

Leave "Auto-assign public IP" on for now — you'll replace it with a stable Elastic IP in step 3, but you need a reachable address to SSH in and finish setup first.

02 Security Groups

Create (or edit) the instance's Security Group with exactly these inbound rules — nothing else.

TYPE	PORT	SOURCE	WHY
SSH	22	Your IP only (x.x.x.x/32)	Never open SSH to the whole internet.
HTTP	80	0.0.0.0/0	Needed for Let's Encrypt's HTTP-01 challenge and the HTTPS redirect.
HTTPS	443	0.0.0.0/0	The actual application traffic.

Do not open 27017 (MongoDB), 6379 (Redis), or 8000 (the backend API) to the internet. All three are reached only from Nginx or the app itself, on `localhost`. If you ever need remote DB access, use an SSH tunnel, not a public inbound rule.

03 Elastic IP

EC2 → Elastic IPs → Allocate, then Actions → Associate → select your instance. This gives you a stable public IP that survives a stop/start (a plain public IP does not) — your DNS record will point here.

04 Domain & DNS configuration

Buy a domain from any registrar (Route 53, Namecheap, GoDaddy — doesn't matter). In its DNS settings, add:

TYPE	NAME	VALUE
A	@ (or <code>crm</code> , if using a subdomain)	Your Elastic IP
A	<code>www</code>	Your Elastic IP (optional, if you want the <code>www.</code> variant to resolve too)

DNS propagation can take a few minutes to a few hours. Confirm it's resolved before step 18 (Let's Encrypt needs your domain to actually point here) — `dig +short your-domain.com` should return your Elastic IP.

05 SSH access

```
# from your local machine
chmod 400 your-key.pem
ssh -i your-key.pem ubuntu@YOUR_ELASTIC_IP
```

06 Update Ubuntu

```
sudo apt update && sudo apt upgrade -y
sudo reboot
```

Wait ~30 seconds, then reconnect with the same SSH command from step 5.

07 Required base packages

```
sudo apt install -y build-essential curl git ufw software-properties-common \
ca-certificates gnupg lsb-release
```

08 Install Python 3.12

Ubuntu 22.04 ships Python 3.10 by default — the backend requires 3.12.

```
sudo add-apt-repository ppa:deadsnakes/ppa -y
sudo apt update
sudo apt install -y python3.12 python3.12-venv python3.12-dev
python3.12 --version    # should print 3.12.x
```

On Ubuntu 24.04, Python 3.12 is already the system default — you can skip the PPA and just `sudo apt install -y python3.12-venv python3.12-dev`.

09 Install Node.js 22

```
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs
```

```
node --version    # v22.x
npm --version
```

10 MongoDB — install and initialize a replica set

The app uses multi-document transactions, which MongoDB only supports on a replica set — even a single-node one.

```
curl -fsSL https://pgp.mongodb.com/server-7.0.asc | sudo gpg -o /usr/share/keyrings/mongo
echo "deb [signed-by=/usr/share/keyrings/mongodb-server-7.0.gpg] https://repo.mongodb.org/
sudo tee /etc/apt/sources.list.d/mongodb-org-7.0.list
sudo apt update
sudo apt install -y mongodb-org
```

Enable the replica set before first start:

```
# /etc/mongod.conf – add under the existing config, or edit if present
sudo sed -i '/^#replication:/a replication:\n  replSetName: "rs0"' /etc/mongod.conf
# if mongod.conf has no #replication: line, append manually instead:
# echo -e "replication:\n  replSetName: rs0" | sudo tee -a /etc/mongod.conf

sudo systemctl enable --now mongod
sudo systemctl status mongod    # confirm "active (running)"

# initialize the single-node replica set (one-time)
mongosh --eval "rs.initiate({_id: 'rs0', members: [{_id: 0, host: 'localhost:27017'}]})"
mongosh --eval "rs.status()"    # should show PRIMARY
```

MongoDB binds to `127.0.0.1` by default (see `bindIp` in `/etc/mongod.conf`) — leave it that way. It should never be reachable from outside this instance.

11 Redis

```
sudo apt install -y redis-server
sudo systemctl enable --now redis-server
redis-cli ping    # should print PONG
```

Redis also binds to localhost only by default (`/etc/redis/redis.conf` , `bind 127.0.0.1`) — leave it that way.

12 Git setup & clone the project

```
# generate a deploy key if the repo is private, or use HTTPS with a PAT
ssh-keygen -t ed25519 -C "deploy@afs-crm" -f ~/.ssh/id_ed25519 -N ""
cat ~/.ssh/id_ed25519.pub # add this as a deploy key in your git host

sudo mkdir -p /opt/afs-crm
sudo chown ubuntu:ubuntu /opt/afs-crm
git clone git@your-git-host:your-org/afs-crm.git /opt/afs-crm
cd /opt/afs-crm
```

13 Environment variables

```
cd /opt/afs-crm/backend
cp .env.production .env
nano .env # fill in every value – see below
```

At minimum, replace every `change-me` with a real value:

```
# generate two DIFFERENT random secrets – never reuse one for both
openssl rand -base64 48 # → JWT_SECRET_KEY
openssl rand -base64 48 # → ENCRYPTION_KEY (must differ from JWT_SECRET_KEY)
```

VARIABLE	SET TO
JWT_SECRET_KEY	the first random value above
ENCRYPTION_KEY	the second random value above
MONGO_URI	mongodb://localhost:27017/?replicaSet=rs0
REDIS_URL	redis://localhost:6379/0
OTP_MODE	production
AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY	a real IAM user/role scoped to just your S3 bucket (see Best Practices)
S3_BUCKET_NAME	your real bucket name

CORS_ORIGINS	https://your-domain.com
FRONTEND_BASE_URL	https://your-domain.com
TRUSTED_PROXY_IPS	127.0.0.1 (Nginx runs on this same box)

The app will refuse to start if `APP_ENV=production` and either secret is missing, too short, or left at the shipped placeholder — this is intentional (a boot-time safety check), not a bug you need to work around.

14 Backend deployment

```
cd /opt/afs-crm/backend
python3.12 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install .
deactivate
```

Quick manual smoke test before wiring up systemd:

```
cd /opt/afs-crm/backend
.venv/bin/uvicorn app.main:app --host 127.0.0.1 --port 8000
# in a second terminal:
curl http://127.0.0.1:8000/api/v1/health
# {"success":true,"data":{"mongo":true,"redis":true,"status":"ok"},...}
# Ctrl+C to stop – systemd will own this process from here on
```

15 Arq worker

Easy to forget, and nothing fails loudly if you do. The worker is a separate long-running process from the API — it handles Meta access-token refresh, reminders, communication sending, and referral-partner background tasks. Without it running, the API still serves requests normally, but those background jobs silently never run.

Uses the same virtualenv as the backend — no separate install needed. Systemd service is set up together with the API in step 19.

```
# manual smoke test
cd /opt/afs-crm/backend
.venv/bin/arq app.worker.worker_settings.WorkerSettings
# should log "Starting worker..." with no errors – Ctrl+C to stop
```

16 Frontend build

```
cd /opt/afs-crm/frontend
npm install
npm run build
# output: /opt/afs-crm/frontend/dist
```

No environment variables are required for a same-origin deployment — the app's API client defaults to the relative path `/api/v1`, which works as long as Nginx (next step) proxies `/api/` to the backend on the same domain.

Re-run `npm run build` after every `git pull` that touches `frontend/` — Nginx serves whatever is currently in `dist/`, it doesn't rebuild automatically.

17 Nginx — install and configure

```
sudo apt install -y nginx
```

```
sudo nano /etc/nginx/sites-available/afs-crm
```

```
server {
    listen 80;
    server_name your-domain.com www.your-domain.com;

    root /opt/afs-crm/frontend/dist;
    index index.html;
    client_max_body_size 20M; # document uploads

    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

```

location /api/ {
    proxy_pass http://127.0.0.1:8000/api/;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
}

```

The `X-Forwarded-For` / `X-Forwarded-Proto` lines are not optional. The backend's proxy-trust middleware reads them to recover the real client IP for rate limiting and audit logs — without them, every request looks like it's coming from this server itself, and rate limits collapse into one shared bucket across every real visitor.

```

sudo ln -s /etc/nginx/sites-available/afs-crm /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t    # "syntax is ok" / "test is successful"
sudo systemctl reload nginx

```

At this point `http://your-domain.com` should load the app (over plain HTTP — SSL is next).

18 SSL certificate (Let's Encrypt)

```

sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d your-domain.com -d www.your-domain.com

```

Certbot edits the Nginx config automatically to add the certificate and an HTTP → HTTPS redirect. Follow its prompts (email address, agree to terms, choose "redirect" when asked).

```

# confirm auto-renewal works (runs via a systemd timer certbot installs)
sudo certbot renew --dry-run

```

Update `CORS_ORIGINS` and `FRONTEND_BASE_URL` in `backend/.env` to the `https://` URL if you haven't already, then restart the backend service (step 19) for the change to take effect.

19 Systemd services & auto-start on reboot

Two services: the API and the worker. Both run as the `ubuntu` user from the same `virtualenv`.

```
sudo nano /etc/systemd/system/afs-crm-backend.service
```

```
[Unit]
Description=AFS CRM Backend API
After=network.target mongod.service redis-server.service

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/opt/afs-crm/backend
ExecStart=/opt/afs-crm/backend/.venv/bin/uvicorn app.main:app --host 127.0.0.1 --port 8000
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

```
sudo nano /etc/systemd/system/afs-crm-worker.service
```

```
[Unit]
Description=AFS CRM Background Worker (arq)
After=network.target redis-server.service

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/opt/afs-crm/backend
ExecStart=/opt/afs-crm/backend/.venv/bin/arq app.worker.worker_settings.WorkerSettings
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now afs-crm-backend afs-crm-worker
```

```
sudo systemctl status afs-crm-backend # confirm "active (running)"
sudo systemctl status afs-crm-worker
```

MongoDB and Redis were already enabled for auto-start in steps 10–11 (`systemctl enable --now`). Nginx enables itself by default on install. After a full `sudo reboot`, all four should come back up on their own — worth testing once now rather than discovering it during a real outage.

20 Firewall (UFW)

```
sudo ufw allow OpenSSH
sudo ufw allow 'Nginx Full' # 80 + 443
sudo ufw enable
sudo ufw status
```

This is a second layer behind the Security Group from step 2 — belt and suspenders. Neither one alone should be your only line of defense.

21 Log locations

WHAT	WHERE
Backend API	<code>journalctl -u afs-crm-backend -f</code>
Worker	<code>journalctl -u afs-crm-worker -f</code>
Nginx access	<code>/var/log/nginx/access.log</code>
Nginx error	<code>/var/log/nginx/error.log</code>
MongoDB	<code>/var/log/mongodb/mongod.log</code>
Redis	<code>journalctl -u redis-server -f</code>

22 Troubleshooting common issues

- **App won't start, log says a secret is missing/too short/placeholder** — expected behavior, not a bug. Fill in a real `JWT_SECRET_KEY` / `ENCRYPTION_KEY` in `.env` (step 13) and restart.
- `/api/v1/health` shows `"mongo": false` — check `sudo systemctl status mongod`, and confirm the replica set actually initialized: `mongosh --eval "rs.status()"` should show a PRIMARY, not an

error.

- **"redis": false** — check `sudo systemctl status redis-server` and that `REDIS_URL` in `.env` matches.
- **502 Bad Gateway from Nginx** — the backend service isn't running or isn't listening on `127.0.0.1:8000`. Check `sudo systemctl status afs-crm-backend` and `journalctl -u afs-crm-backend -n 50`.
- **Frontend loads but every API call fails** — usually a stale `dist/` build or a wrong `CORS_ORIGINS`. Rebuild the frontend (step 16) and confirm `CORS_ORIGINS` matches the exact URL you're visiting (including `https://`).
- **Login/OTP rate limits trip almost immediately for everyone** — the proxy-trust headers aren't wired correctly. Re-check the `X-Forwarded-For` / `X-Forwarded-Proto` lines in the Nginx config (step 17) and that `TRUSTED_PROXY_IPS=127.0.0.1` in `.env`.
- **Meta webhook / integration test never succeeds** — confirm the worker service is actually running (step 15/19 — it's the process most commonly forgotten) and check its logs for the specific Graph API error.
- **Let's Encrypt fails with a challenge error** — confirm DNS has actually propagated (`dig +short your-domain.com` returns your Elastic IP) and port 80 is open in the Security Group before retrying Certbot.

23 Backup recommendations

- **MongoDB:** a nightly `mongodump` to a local directory, synced to S3 (a separate bucket from documents, or a distinct prefix with its own lifecycle rule). A daily EBS snapshot of the whole volume is a reasonable simpler alternative.
- **S3 documents:** enable bucket versioning so an accidental overwrite/delete is recoverable.
- **Test the restore, not just the backup** — an untested backup is a hope, not a plan. Restore a `mongodump` into a scratch database at least once and confirm the app can read it.

```
# simple daily dump, add to root's crontab via `sudo crontab -e`
0 2 * * * mongodump --uri="mongodb://localhost:27017/afs_crm?replicaSet=rs0" \
  --out=/opt/backups/mongo/$(date +%F) && \
  aws s3 sync /opt/backups/mongo/$(date +%F) s3://your-backup-bucket/mongo/$(date +%F)/
```

24 Monitoring recommendations

- **Uptime:** an external check (CloudWatch Synthetics, or any third-party uptime monitor) against `https://your-domain.com/api/v1/health`, alerting on anything other than a 200 with

`"status": "ok"` .

- **Disk space:** CloudWatch agent + an alarm at 80% used — MongoDB data and logs both grow continuously and will eventually fill the volume if unwatched.
- **CPU/memory:** standard CloudWatch EC2 alarms are enough at Phase 1 scale.
- **Service health:** systemd's own `Restart=on-failure` (already configured in step 19) handles transient crashes; pair it with an alert on repeated restarts (`systemctl status` shows restart count) so a crash loop doesn't go unnoticed.

25 Deployment verification checklist

Run through this in a real browser against the live HTTPS URL before calling the deployment done. Full detail in the companion AWS Deployment Checklist — abbreviated here:

- Owner can log in over HTTPS with a valid padlock.
- A test lead can be created, assigned, and viewed.
- A secure application link can be generated, opened, logged into, filled, and submitted end-to-end.
- A document upload succeeds and the file is reachable in S3.
- `GET /api/v1/health` returns `"status": "ok"` .
- Non-Phase-1 nav items (Tasks, Reports, Message Center, Administration, Referral Partners, most of Settings) show "Soon" and their direct URLs resolve to a Coming Soon page, not the real feature.
- A reboot test (`sudo reboot`) brings Mongo, Redis, the backend, the worker, and Nginx all back up without manual intervention.

26 Production best practices

- **Prefer an IAM role over long-lived S3 keys.** Attach an instance profile with a policy scoped to just your document bucket instead of putting `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` in `.env` — the app's S3 client already works with either.
- **Never commit a populated `.env` file.** The repo's `.env.example` / `.env.production` are templates with placeholders by design — keep it that way.
- **Rotate `JWT_SECRET_KEY` and `ENCRYPTION_KEY` independently.** They're deliberately separate secrets specifically so rotating one (e.g. after a suspected token leak) never makes the other's encrypted data (stored integration credentials) unreadable.
- **Keep the worker and API as separate systemd services,** not one combined process — it makes resource limits, restart behavior, and log triage independent for each.

- **Rebuild the frontend on every deploy that touches it** — Nginx serves a static snapshot, not a live-rendering server; a backend-only deploy doesn't need a frontend rebuild, but a frontend change is invisible until you run `npm run build` again.
- **Record the git commit you deployed** before pulling the next one, so a rollback is a known, rehearsed action rather than a scramble.
- **A real SMS provider must be wired in before this is a working OTP flow in production** — `OTP_MODE=production` alone doesn't send anything; it just stops using the fixed development OTP. Configure the SMS client before relying on forgot-password/signup OTP delivery for real users.
- **Schedule a formal load test and a penetration test against a staging copy** before wider rollout — neither is something a code-level audit can substitute for.

AFS Financial CRM — AWS Deployment Guide · see also the Phase 1 Readiness/Test/Security Report and the AWS Deployment Checklist.